

Dentro do Transformer

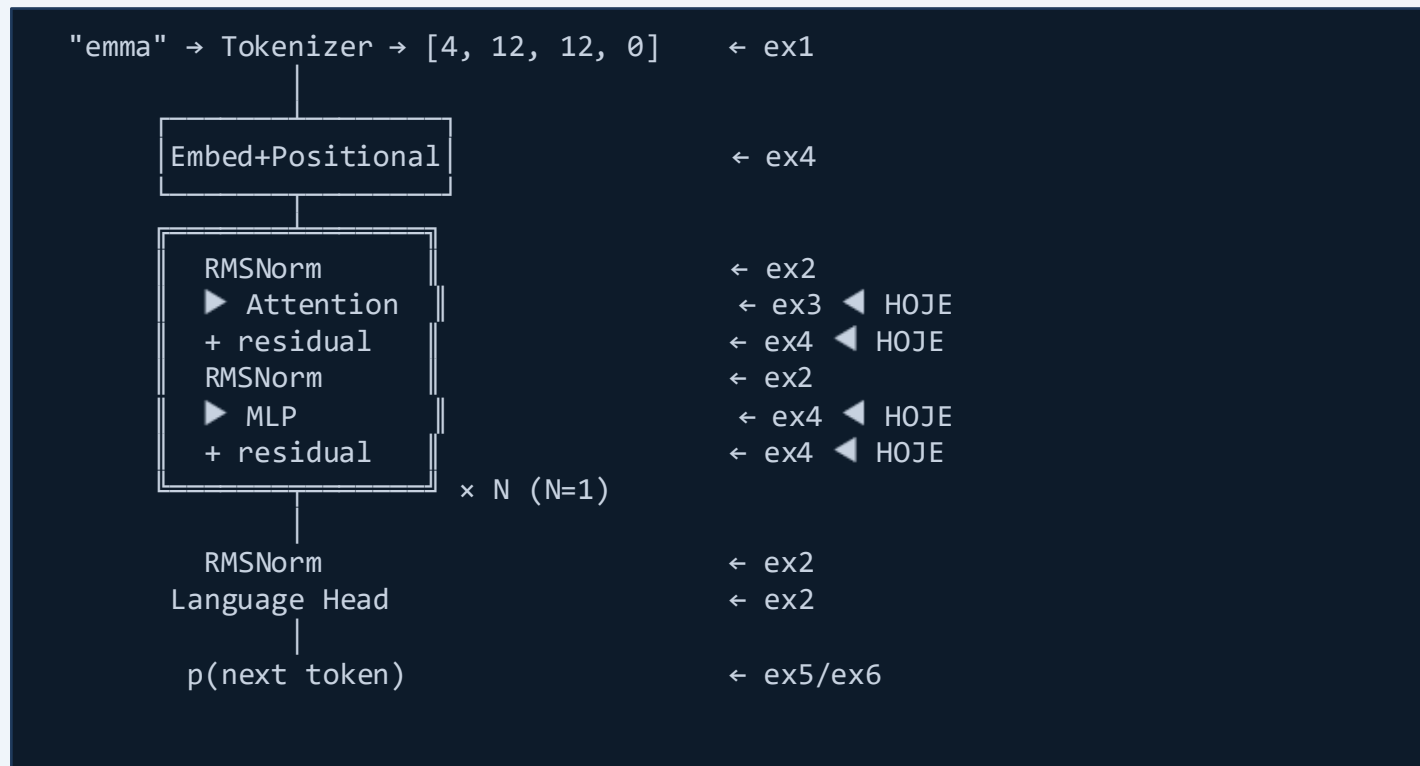
Self-Attention

SGAIM — Semana 3

Semana 1: $p(x)$, manifolds · Semana 2: tokenização, arquiteturas, backprop

Hoje: o coração do Transformer — self-attention.

O mapa do Transformer — onde estamos hoje



Ex3: self-attention (single head) · Ex4: modelo completo (multi-head, MLP, residual)

Progresso: ex1 e ex2

Antes de avançar — sondagem rápida:

- Quem terminou o ex1 (todos os testes passam)?
- Quem terminou o ex2?
- Quem ficou bloqueado? Onde?

Se ficaram bloqueados no TODO 3 do ex1 (backward): não estão sozinhos — é o mais difícil.

Se ficaram bloqueados no Test 3 do ex2 (softmax): releiam o comentário do teste.

- **Prioridade: ex1 e ex2 completos esta semana.**

O problema: a mesma palavra, significados diferentes

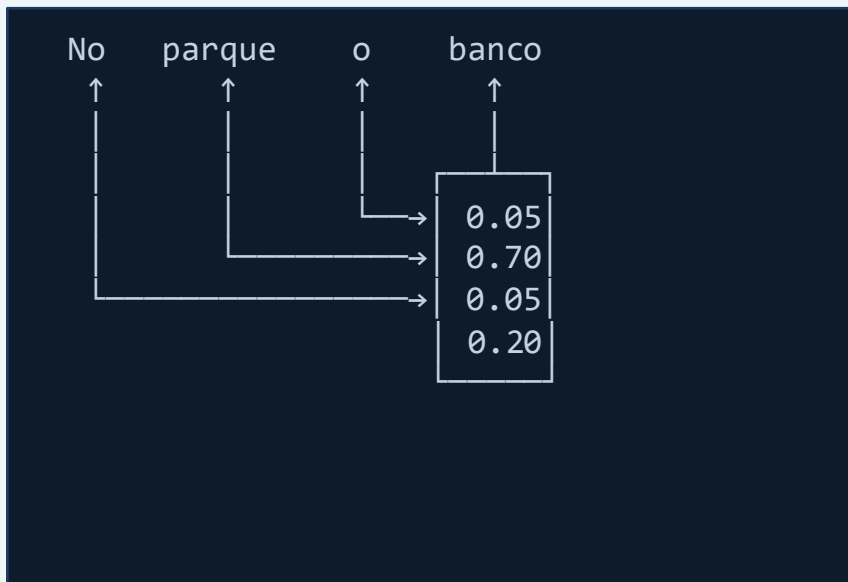
"O banco estava cheio."

Banco = assento no parque? Instituição financeira?

- **O significado depende do contexto — das palavras que vêm antes.**
- Uma embedding fixa (lookup table) atribui o mesmo vector a "banco" em ambos os casos.
- Static embedding — um vector por palavra, independente do contexto. Não basta.
- **Precisamos de um mecanismo que ajuste a representação de cada token com base no contexto.**

A ideia central: attention

E se cada token pudesse olhar para todos os tokens anteriores e decidir quais são relevantes?



- "banco" distribui pesos de atenção sobre os tokens anteriores.
- "parque" recebe 70% — desambigua "banco" como assento.
- **Isto é attention. O token decide a quem prestar atenção.**
- O resultado é um contextual embedding — o vector de "banco" move-se em R^n conforme o contexto.

Três perguntas por token: Q, K, V

Para decidir a quem prestar atenção, cada token gera 3 vectores:

Vector	Pergunta	Analogia: biblioteca
Q (Query)	O que procuro?	A pergunta que fazes ao bibliotecário
K (Key)	O que ofereço?	O índice do livro
V (Value)	O que comunico?	O conteúdo do livro

- O token actual gera um Q. Cada token passado tem um K e um V.
- Comparamos Q com cada K → scores de relevância.
- Usamos os scores para pesar os V → obtemos a informação útil.

Projecções lineares

Os vectores Q, K, V são projecções aprendidas do embedding:

```
Q = linear(x, W_Q)      # o que procuro?  
K = linear(x, W_K)      # o que ofereço?  
V = linear(x, W_V)      # o que comunico?
```

- Três matrizes de pesos (W_Q , W_K , W_V), cada uma de dimensão $n_{\text{embd}} \times n_{\text{embd}}$.
- São as "lentes" através das quais o token vê o contexto.
- O modelo aprende que lentes usar durante o treino.

No ex3: `state_dict['attn_wq'], state_dict['attn_wk'], state_dict['attn_wv']`.

Score de atenção

Quanto é que o token actual deve prestar atenção ao token na posição t ?

$$\text{score}(q, k_t) = (q \cdot k_t) / \sqrt{d}$$

- $q \cdot k_t$ = produto interno — mede similaridade entre o que procuro (Q) e o que o token t oferece (K).
- Quanto mais alinhados Q e K, maior o score.
- **$\sqrt{d}$ = scaling factor. Porquê?**

THINK: porque dividir por $\sqrt{d}$?

O que acontece ao softmax se não dividirmos pelo $\sqrt{d}$?

- Pensem 2 minutos. Considerem $d = 64$.
- Se cada componente do dot product contribui ~ 1 , o resultado é ~ 64 .
- **O que é que o softmax faz com inputs de magnitude 64?**

O scaling previne a saturação

Sem scaling, com $d = 64$:

```
Dot products têm magnitude  $\sim \sqrt{64} = 8$   
softmax([8, 0, 0, ...])  $\rightarrow$  [0.9997, 0.0001, ...]  
 $\rightarrow$  0 softmax satura – quase toda a massa vai para um token.  
 $\rightarrow$  Gradientes  $\approx 0 \rightarrow$  o modelo não aprende. (Mesma ideia do overflow no ex2!)
```

Com scaling:

```
Dividir por  $\sqrt{64} = 8 \rightarrow$  scores na escala  $\sim 1$   
softmax([1, 0, 0, ...])  $\rightarrow$  [0.58, 0.21, 0.21]  
 $\rightarrow$  Gradientes saudáveis. O modelo aprende a distribuir atenção.
```

Softmax → pesos de atenção

$$\text{weights} = \text{softmax}(\text{scores})$$

Os scores tornam-se uma distribuição de probabilidade: cada peso ≥ 0 , soma = 1.

Perspectiva de "cheio" (último token, vê todos):

Token:	"No"	"parque"	"o"	"banco"	"estava"	"cheio"
Score:	0.1	0.5	0.1	2.0	2.5	0.3
			↓ softmax			
Weight:	0.05	0.07	0.04	0.30	0.49	0.05
			↑	↑		
			estes tokens recebem mais atenção de "cheio"			

É a mesma softmax que implementaram no ex2!

Output: média ponderada dos valores

$$\text{output} = \sum_t \text{weights}_t \cdot v_t$$

Cada token t contribui com o seu valor (v), ponderado pelo peso de atenção.

```
output_"cheio" = 0.05·v_"No" + 0.07·v_"parque" + ...  
                + 0.30·v_"banco" + 0.49·v_"estava" + 0.05·v_"cheio"
```

- O token "absorve" informação dos tokens relevantes.
- A sua representação fica enriquecida pelo contexto — o contextual embedding em acção.

É isto que o ex3 TODO 4 implementa — uma média ponderada de vectores.

THINK: atenção na posição 0

**Quando o token na posição 0 calcula atenção,
quantos tokens pode ver?**

E quais serão os pesos de atenção?

Máscara causal

- Posição 0: 1 token (ele próprio). Pesos = $[1.0]$.
- Posição 1: 2 tokens. Pesos = $[w_0, w_1]$, soma = 1.
- Posição 2: 3 tokens. Pesos = $[w_0, w_1, w_2]$, soma = 1.

Posição →	0	1	2	3	4
Vê 0:	✓				
Vê 1:	✓	✓			
Vê 2:	✓	✓	✓		
Vê 3:	✓	✓	✓	✓	
Vê 4:	✓	✓	✓	✓	✓

- Attention causal — cada token só vê o passado e a si próprio. Nunca o futuro.
- É o que torna o modelo generativo (decoder-only).

No ex3, a causalidade é automática: a lista keys só tem tokens até à posição actual.

Causal vs bidireccional

	GPT (causal)	BERT (bidireccional)
Vê	passado + si próprio	toda a sequência
Útil para	geração de texto	classificação, sentence embedding, search
Vocês implementam	✓ (ex3, ex4)	—

- **Consequência da causalidade: o último token é o mais rico — viu todos os anteriores.**
- Após N layers, a sua representação codifica o sentido da frase inteira.
- É por isso que o usamos para prever o token seguinte.

KV Cache: eficiência na geração

Quando geramos texto token a token, K e V dos tokens anteriores não mudam.
Não os recalculamos — guardamos em cache:

```
Pos 0: keys = [k0],          values = [v0]          → 1 entrada  
Pos 1: keys = [k0, k1],      values = [v0, v1]      → 2 entradas  
Pos N: keys = [k0, ..., kn], values = [v0, ..., vn] → N+1 entradas
```

- No ex3, o KV cache são as listas keys e values que crescem a cada chamada.
- O Test 3 verifica que cresce correctamente: 1, 2, 3 posições.

Resumo: single-head attention

```
x (embedding + posição)
├── Q = linear(x, W_Q)  ─── "o que procuro?"
├── K = linear(x, W_K)  ─── "o que ofereço?" → cache
└── V = linear(x, W_V)  ─── "o que comunico?" → cache

scores = (Q · cada K) / √d
weights = softmax(scores)      ← soma = 1
head_out =  $\sum \text{weights}_t \cdot V_t$   ← média ponderada
output = linear(head_out, W_O) ← projecção final
```

4 operações
4 TODOs no ex3

Isto é
self-attention.
O coração do
Transformer.

Multi-head attention: 4 perspectivas

Em vez de 1 "perspectiva", usamos 4 heads. Cada head opera em $d/4$ dimensões ($\text{head_dim} = \text{n_embd} / \text{n_head} = 16/4 = 4$).

Analogia: ler uma frase para gramática, significado, tom, referência — 4 leituras paralelas, cada uma especializada.

```
n_embd = 16  
n_head = 4  
head_dim = 4
```

```
Q, K, V: dim 16 cada
```

```
Head 0: dims [0:4]    Head 1: dims [4:8]  
Head 2: dims [8:12]   Head 3: dims [12:16]
```

Cada head tem a mesma mecânica do ex3, mas em 4 dimensões em vez de 16.

Concatenação e projecção

Cada head produz um vector de dimensão 4. Concatenamos:

```
head0 (dim 4) ++ head1 (dim 4) ++ head2 (dim 4) ++ head3 (dim 4) = dim 16
```

Depois, uma projecção final (W_O) mistura a informação dos 4 heads:

```
x_attn = linear(concat, W_O)      # dim 16 → dim 16
```

- Cada head descobre padrões diferentes.
- A projecção final combina-os num único vector de saída.

THINK: porquê multi-head?

**Porque não usar 1 head com dim 16
em vez de 4 heads com dim 4?**

Se 1 head com dim 16 tem o mesmo número de parâmetros,
qual é a vantagem de dividir em 4 heads?

Múltiplos padrões de atenção

1 head com dim 16 → 1 distribuição de pesos por posição.

4 heads com dim 4 → 4 distribuições diferentes por posição.

```
Head 0: "banco" atende a "parque"    (semântica – desambiguação)
Head 1: "banco" atende a "o"         (gramática – artigo adjacente)
Head 2: "banco" atende a "No"        (posição – início de frase)
Head 3: "banco" atende a "banco"     (auto-referência)
```

- **4 revisores especializados > 1 generalista.**
- Cada head aprende a focar em diferentes tipos de relações.
- Isto só é possível com pesos separados por head.

Residual connections

Depois do attention, somamos o input original:

```
x_out = x + attention(x)
```

THINK: porque somar o input ao output? O que muda nos gradientes?

A auto-estrada dos gradientes

Sem residual:

$x \rightarrow f(x) \rightarrow g(f(x)) \rightarrow \dots$

$\text{grad} = g' \cdot f' \cdot \dots$ $\leftarrow$ pode desaparecer (vanishing gradients)

$$\partial(x + f(x)) / \partial x = 1 + f'(x)$$

Com residual:

$x \rightarrow x + f(x) \rightarrow x + f(x) + g(x + f(x)) \rightarrow \dots$

$\text{grad} = 1 + f' + \dots$ $\leftarrow$ mínimo 1 — nunca desaparece

O gradiente tem sempre o termo 1. É uma "auto-estrada" directa do output até ao input. Sem residual, redes profundas não treinam.

No ex4: TODO 2 e TODO 4 — $x = [x_i + r_i \text{ for } x_i, r_i \text{ in zip}(x, x_residual)]$.

RMSNorm: reminder rápido

Normaliza a magnitude do vector — divide pela RMS (Root Mean Square):

$$\text{rmsnorm}(\mathbf{x})_i = \mathbf{x}_i / \sqrt{ (1/d) \cdot \sum_j \mathbf{x}_j^2 + \epsilon }$$

- Sem normalização → valores crescem ou diminuem descontroladamente entre camadas.
- Já implementaram no ex2.
- No modelo completo, aparece 3 vezes: antes do attention, antes do MLP, antes do language head.

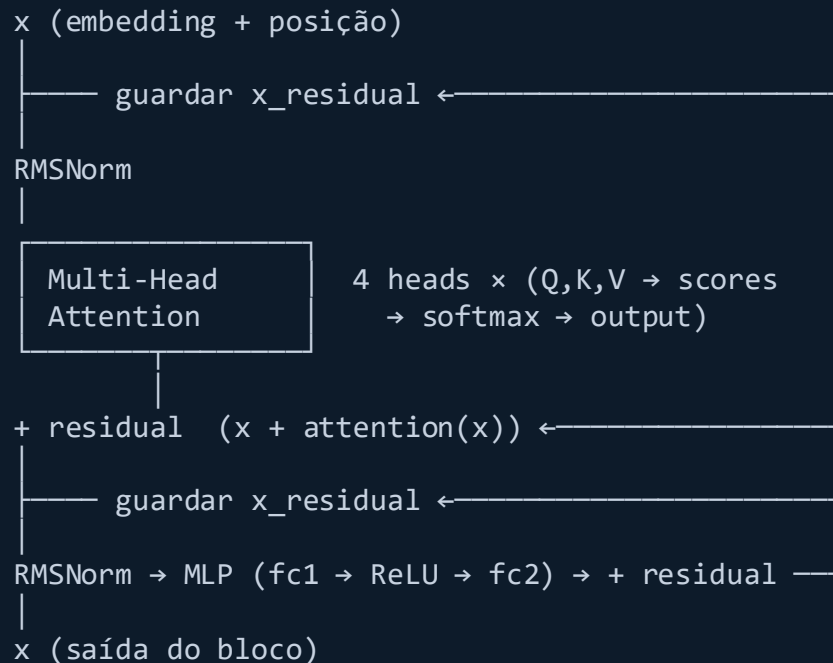
MLP block: expansão e compressão

Duas camadas lineares com ReLU entre elas:

```
x → linear(fc1) → ReLU → linear(fc2) → output
   16 → 64      max(0,x)   64 → 16
   (expansão 4×)          (compressão)
```

- A expansão dá ao modelo espaço para transformações não-lineares mais ricas.
- A compressão traz de volta à dimensão original.
- No ex4: TODO 3. Usar `state_dict['layer0.mlp_fc1']` e `state_dict['layer0.mlp_fc2']`.
- **THINK: onde vivem mais parâmetros — attention ou MLP?**
- **MLP: $16 \times 64 + 64 \times 16 = 2048$. Attention: $4 \times 16 \times 16 = 1024$. O MLP tem o dobro!**

O bloco Transformer completo



Isto é o ex4.

Cada TODO
corresponde a
uma secção
deste diagrama.

Contagem de parâmetros

Componente	Cálculo	Parâmetros
Token embeddings (wte)	27×16	432
Position embeddings (wpe)	16×16	256
Attention (Q, K, V, O)	$4 \times 16 \times 16$	1024
MLP (fc1 + fc2)	$16 \times 64 + 64 \times 16$	2048
Language head (lm_head)	27×16	432
Total		4 192

- **4.192 parâmetros. Algoritmicamente idêntico ao GPT-2/3/4.**
- A diferença é escala: dims maiores, mais layers, mais dados. A arquitectura é a mesma.
- O Test 4 do ex4 verifica: `assert len(params) == 4192`.

Roteiro hands-on: ex3 e ex4

Ordem: ex3 primeiro (single-head attention), depois ex4 (modelo completo).

O ex3 prepara directamente o ex4 — a lógica de attention é reutilizada.

```
Ex3: 4 TODOs → single-head attention
```

```
Ex4: 4 TODOs → multi-head + MLP + residual + forward
```

- Trabalhem em pares.
- Quando ficarem bloqueados: olhem para o diagrama (slides 17 e 26).
- **THINK no papel antes de CODE no editor.**
- Especialmente no ex3 TODO 2 (scores).

Ex3 — O que implementar

4 TODOs em `single_head_attn()`:

TODO	O quê	Dica
1	Projecções Q, K, V	<code>linear(x, state_dict['attn_wq'])</code> — depois append K e V ao cache
2	Scores de atenção	Dot product de Q com cada K no cache, dividido por <code>vhead_dim</code>
3	Softmax	Reutilizar a função softmax do ex2 — já está provided
4	Soma ponderada	Para cada dim j: <code>sum(w * v[j] for w, v in zip(weights, values))</code>

Testes: output dim correcta · pesos somam 1.0 · KV cache cresce: 1, 2, 3 · token 0: 100% atenção em si · tokens diferentes → outputs diferentes

Ex4 — O que implementar

4 TODOs em gpt():

TODO	O quê	Dica
1	Multi-head attention	Split Q,K,V em 4 slices de dim 4, attention em cada, concat
2	Residual (attention)	$x = [x_i + r_i \text{ for } x_i, r_i \text{ in zip}(x, x_residual)]$
3	MLP	linear(fc1) → relu cada elemento → linear(fc2)
4	Residual (MLP)	Igual ao TODO 2

Testes: 27 logits · sequência de 3 tokens correcta · tokens diferentes → outputs diferentes · param_count = 4192 · gradientes fluem até às embeddings

Referência durante o hands-on

Vão ter projectados dois diagramas de referência durante o trabalho:

- Slide 17 — Single-head attention → para o ex3
- Slide 26 — Bloco Transformer completo → para o ex4
- Cada TODO tem uma seta no diagrama.

"Onde é que eu estou no diagrama? O que entra? O que sai?"

Wrap-up

O que construíram (e vão construir) hoje:

- Semana 2: motor de gradientes (ex1) + building blocks (ex2)
- **Semana 3: attention (ex3) + modelo completo (ex4)**

Temos o motor montado. Falta ligar a ignição.

Semana 4: treino e geração

Na próxima semana:

- Ex5: cross-entropy loss, SGD, Adam — ver a loss descer de 3.30 para valores úteis
- Ex6: geração de nomes + ablation studies (remover componentes e medir impacto)
- **Ex6 é o core da avaliação. As explicações valem metade da nota.**
- **Entrega do MP1: semana 5.**

TPC

- **Mínimo: completar ex3 e ex4 (todos os testes passam). Responder THINK e REFLECT.**
- Recomendado: EXPLORE do ex3 (remover scaling — o que acontece?) e ex4 (remover residual, mudar n_head).
- Leitura opcional: Jay Alammar, "The Illustrated Transformer" — secção de multi-head attention.